



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

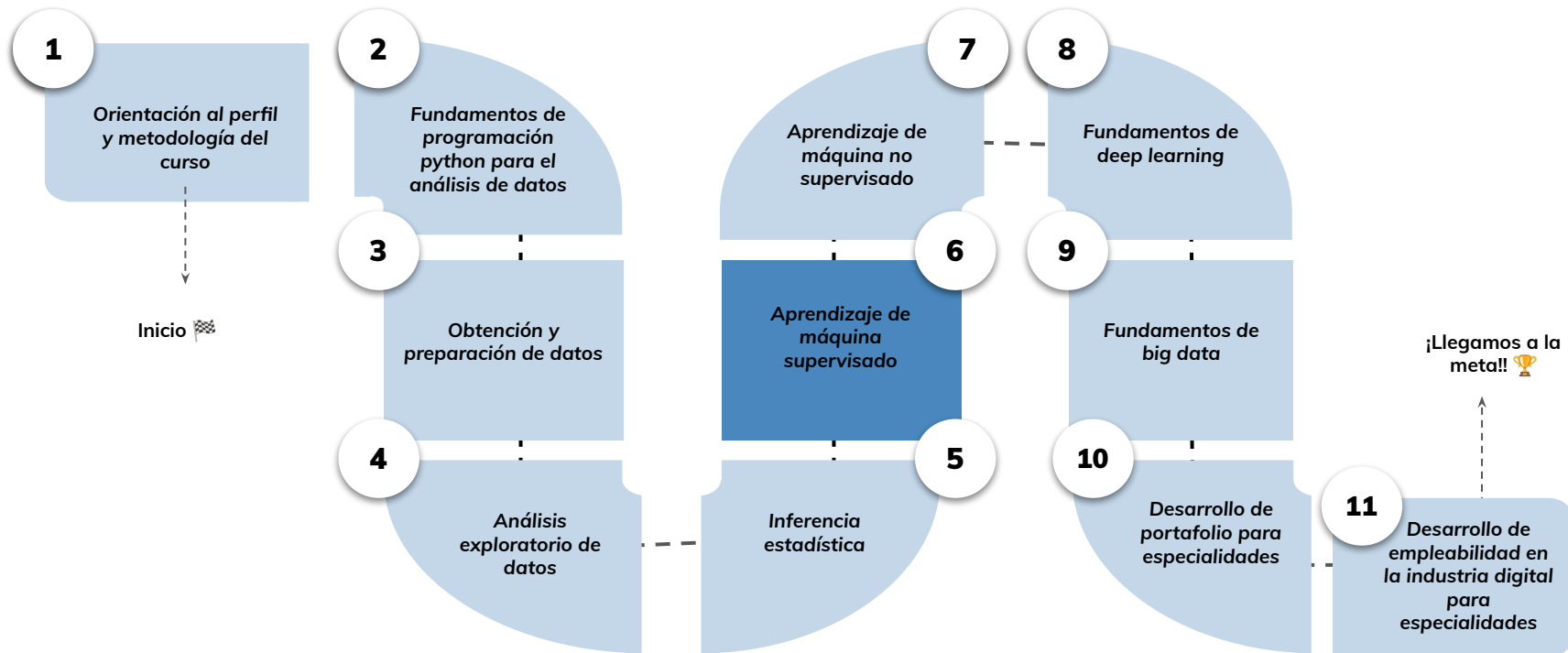


› Algoritmos de boosting- Parte I

Aprendizaje Esperado 8: Elaborar un modelo predictivo aplicando técnicas de boosting para resolver un problema de clasificación utilizando el lenguaje python.

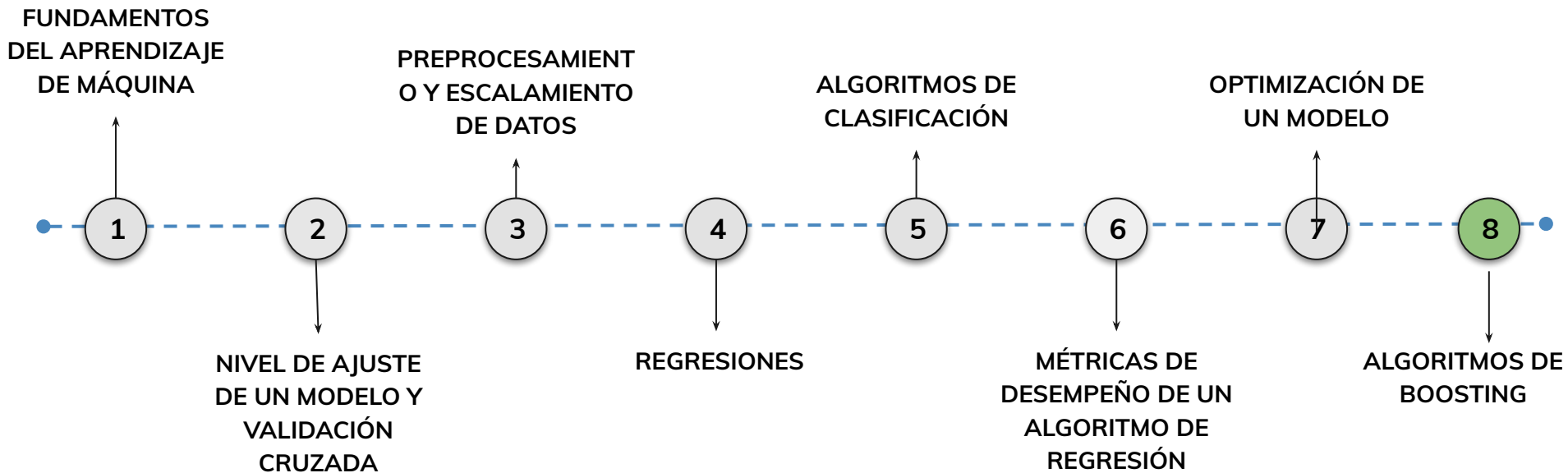
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

8.

Introducción al Boosting

qué es, por qué es útil y cómo se diferencia del bagging.

¿Qué es el boosting?

Lógica secuencial de mejora de errores

Modelo fuerte a partir de modelos débiles

Bagging vs. Boosting

Diferencias clave en entrenamiento y resultados

Casos de uso y comparación

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender el funcionamiento de los algoritmos de boosting.
- Diferenciar entre Bagging y Boosting a nivel conceptual y técnico.
- Implementar modelos de Gradient Boosting en Python con Scikit-learn.
- Evaluar métricas y riesgos de sobreajuste en boosting.
- Aplicar validaciones y ajuste de hiperparámetros para mejorar el rendimiento.

Repaso clase anterior

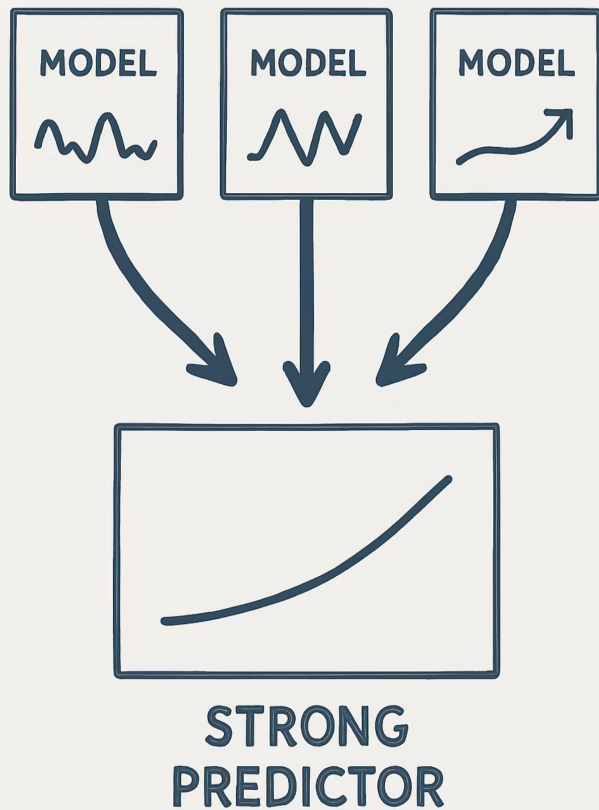
¿Quedó alguna duda?

En la clase anterior trabajamos :

- Implementación de pipelines de optimización con Optuna.
- Comparación entre Grid Search y optimización bayesiana.
- Aplicación de técnicas de ensamblado como Stacking y Bagging.
- Interpretabilidad de modelos con SHAP.

Algoritmos de boosting

MACHINE LEARNING ENSEMBLE METHODS



Algoritmos de Boosting en Aprendizaje Automático

En la actualidad, los modelos de aprendizaje automático desempeñan un papel esencial en la solución de problemas complejos en áreas como finanzas, salud, comercio electrónico y más. Entre las técnicas más eficaces se encuentran los métodos de ensamblado (ensemble), dentro de los cuales destacan los algoritmos de boosting.

Importancia del Boosting

Estas técnicas permiten combinar múltiples modelos débiles para formar uno fuerte, mejorando la capacidad predictiva frente a otros enfoques individuales.

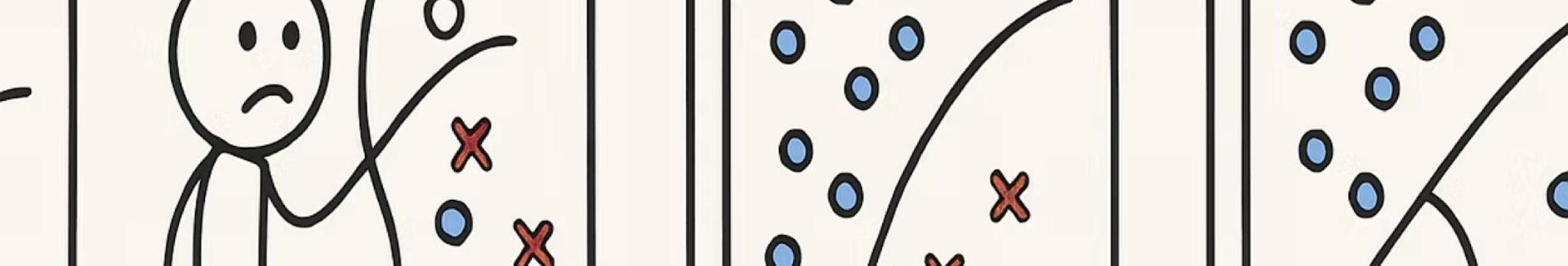
Boosting ha demostrado ser especialmente útil en contextos donde los datos presentan ruido, outliers o una estructura difícil de modelar con técnicas tradicionales.

Su enfoque secuencial y correctivo lo convierte en una herramienta muy eficaz para resolver problemas de clasificación y regresión con altos niveles de precisión.

Aplicaciones Prácticas

Es por ello que se ha convertido en un estándar en muchas competencias de ciencia de datos y aplicaciones industriales.

Los algoritmos de boosting son ampliamente utilizados en diversos sectores donde se requiere alta precisión predictiva.



¿En qué consiste el Boosting?

El boosting es una técnica de aprendizaje automático basada en la idea de construir un modelo fuerte a partir de múltiples modelos débiles. A diferencia de otros enfoques ensemble, el boosting trabaja de forma secuencial, lo que significa que cada modelo nuevo intenta corregir los errores del anterior.

Esto se traduce en una mejora progresiva de la capacidad predictiva general del sistema.

Lógica del Boosting

Primer modelo

Primero se entrena un modelo sobre los datos

Nuevos modelos

Se entrena un nuevo modelo que se enfoca en estos ejemplos difíciles

Análisis de errores

Se analiza qué ejemplos fueron clasificados erróneamente y se les asigna un mayor peso

Modelo final

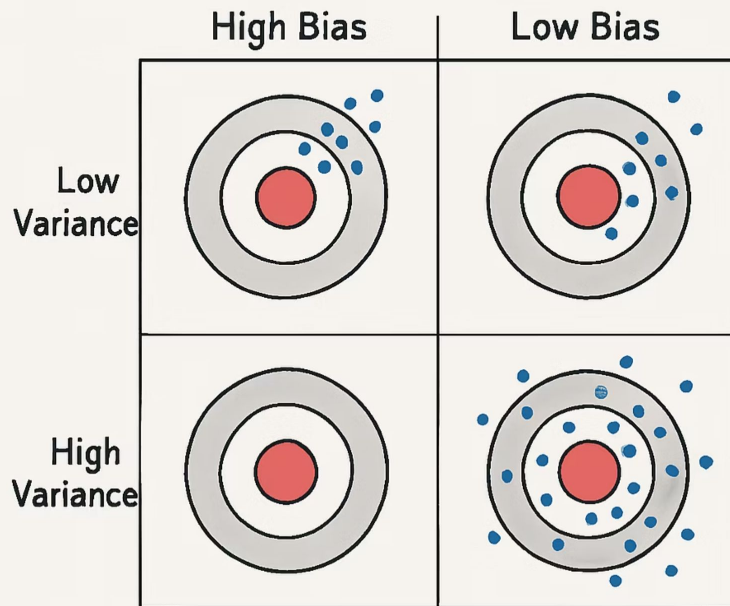
Se van acumulando modelos especializados que en conjunto ofrecen una predicción final más precisa

Eficacia del Boosting

Una de las razones por las que boosting es tan eficaz radica en que reduce el sesgo del modelo, es decir, su capacidad para captar relaciones complejas en los datos.

Mientras que otras técnicas, como el bagging, se enfocan más en reducir la varianza, boosting ofrece una solución potente cuando los modelos individuales por sí solos no logran buenos resultados.

MODEL BIAS AND VARIANCE



Algoritmos Principales de Boosting

AdaBoost

El algoritmo de boosting más conocido y uno de los primeros en desarrollarse.

Gradient Boosting

Evolución que utiliza el descenso de gradiente para minimizar funciones de pérdida.

XGBoost

Implementación optimizada con regularización adicional.

LightGBM

Versión más rápida que utiliza árboles basados en histogramas.

CatBoost

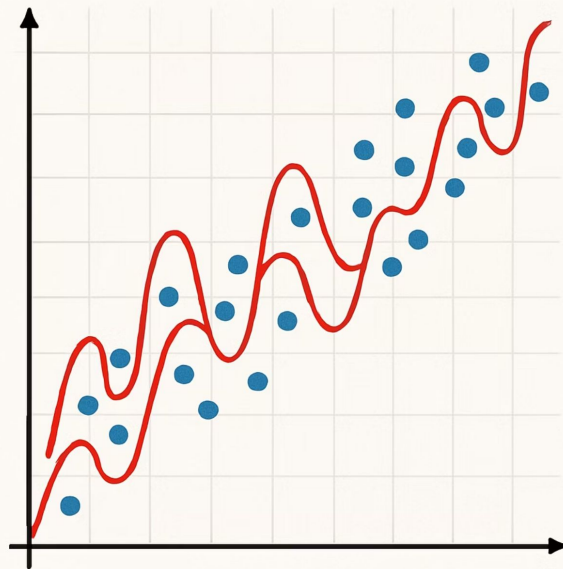
Especializado en el manejo eficiente de variables categóricas.

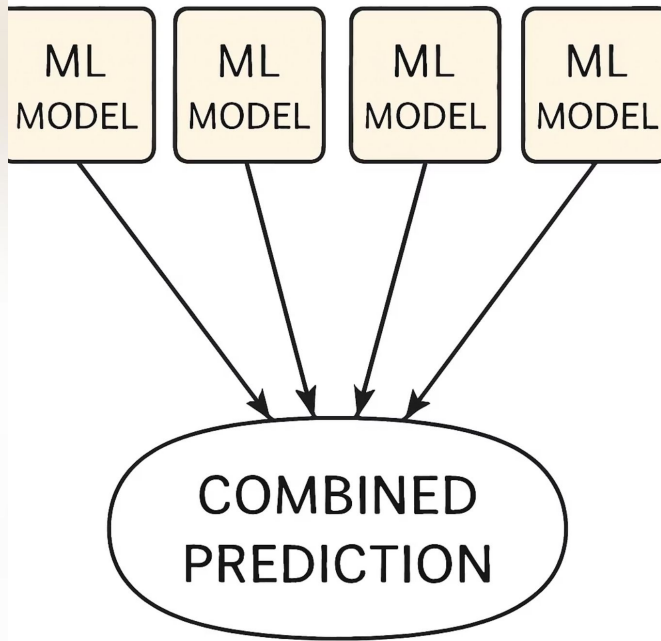
Limitaciones del Boosting

Es importante señalar que el boosting también tiene limitaciones, especialmente en presencia de ruido o datos mal etiquetados. Como el algoritmo presta especial atención a los errores, puede terminar sobreajustando si se le permite entrenar por demasiadas iteraciones.

Por eso, su aplicación debe incluir estrategias de regularización y validación.

OVERFITTING IN MACHINE LEARNING MODELS





Métodos Ensemble

El término ensemble hace referencia a una estrategia general en la que se combinan varios modelos de predicción con el fin de mejorar el rendimiento global.

La lógica detrás de este enfoque es simple: al integrar múltiples perspectivas, se obtiene una visión más completa y robusta que con un solo modelo.

Estrategias Principales de Ensemble

Bagging

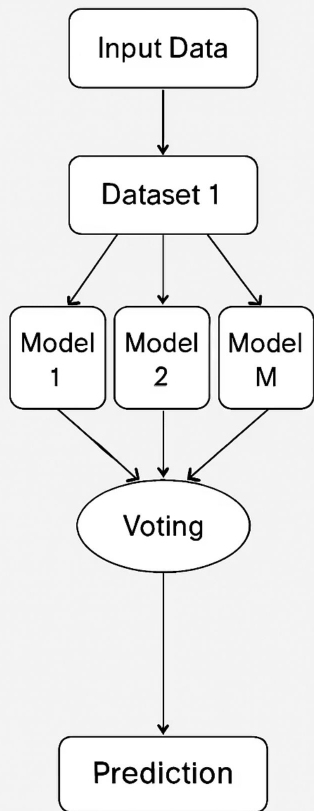
Como su nombre indica (bootstrap aggregating), se basa en generar múltiples subconjuntos aleatorios del conjunto de datos, entrenar modelos de forma paralela sobre cada uno, y luego combinar sus predicciones. Su principal ventaja es la reducción de la varianza.

Boosting

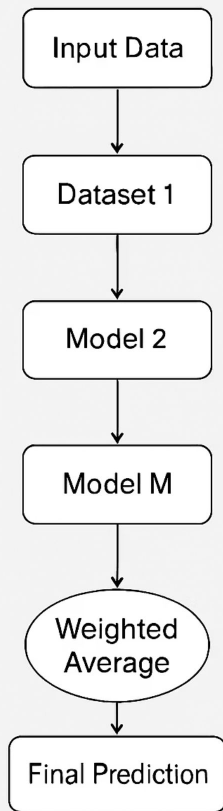
Construye modelos secuencialmente. Cada nuevo modelo aprende a corregir los errores cometidos por el modelo anterior, lo que reduce el sesgo. Este enfoque es más complejo computacionalmente, pero también más preciso en muchos casos.

Por ello, es frecuente ver a boosting superar a bagging en tareas de clasificación complejas.

BAGGING



BOOSTING

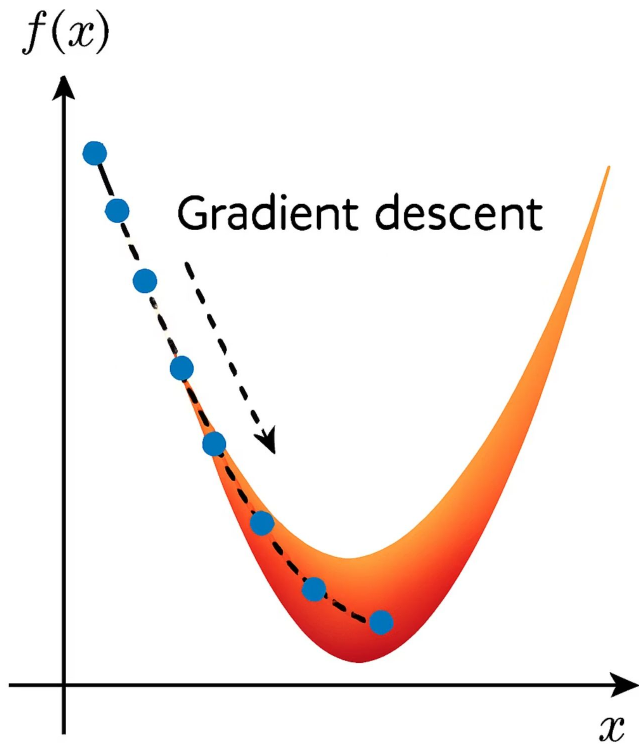


Diferencias Clave entre Bagging y Boosting

Otra diferencia clave entre ambos es que bagging asigna el mismo peso a todas las observaciones, mientras que boosting ajusta dinámicamente el peso de cada ejemplo en función de su dificultad para ser clasificado correctamente. Esto da lugar a una mayor sensibilidad del modelo a casos difíciles, pero también a una mayor posibilidad de sobreajuste si no se regula adecuadamente.

Comparación de Bagging y Boosting

Característica	Bagging	Boosting
Entrenamiento	Paralelo	Secuencial
Objetivo principal	Reducir varianza	Reducir sesgo
Sensibilidad a ruido	Menor	Mayor
Peso de ejemplos	Uniforme	Ajustado por error
Ejemplo clásico	Random Forest	Gradient Boosting



El algoritmo Gradient Boosting

El algoritmo Gradient Boosting es una evolución del enfoque de boosting tradicional que utiliza una técnica conocida como **descenso de gradiente** para minimizar una función de pérdida.

En lugar de simplemente enfocarse en las instancias mal clasificadas, como hace AdaBoost, Gradient Boosting ajusta el modelo para minimizar directamente el error, guiándose por el gradiente de la función de pérdida respecto a las predicciones actuales.

Funcionamiento del Gradient Boosting

Construcción de modelo

En cada iteración, se construye un nuevo modelo (típicamente un árbol de decisión poco profundo) que predice los residuos o errores cometidos por el conjunto actual.

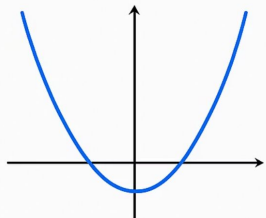
Ajuste con peso

Se suma este nuevo modelo al conjunto anterior con un peso ajustado mediante un parámetro de aprendizaje, también llamado `learning_rate`.

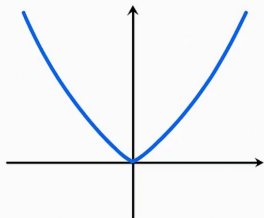
Correcciones suaves

Esto permite hacer correcciones suaves que evitan el sobreajuste si se ajustan adecuadamente.

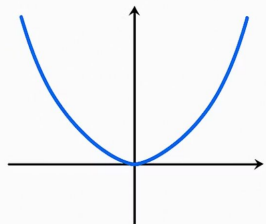
Different Loss Functions



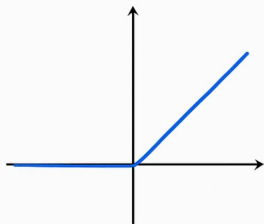
Mean Squared
Error



Mean Absolute
Error



Huber Loss



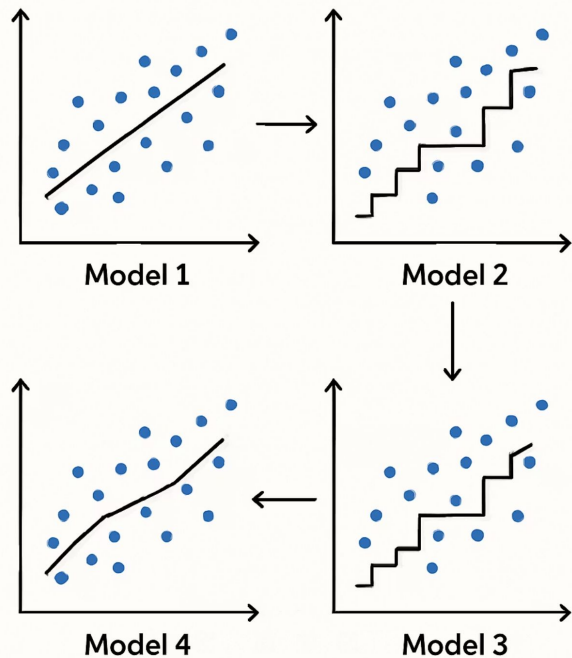
Hinge Loss

Flexibilidad del Gradient Boosting

Uno de los puntos fuertes del Gradient Boosting es su flexibilidad. Puede emplear diferentes funciones de pérdida (error cuadrático medio, log loss, etc.) dependiendo de si se trata de un problema de regresión o clasificación.

Además, se puede regular mediante hiperparámetros como `n_estimators`, `max_depth`, `learning_rate` y `subsample`, lo que brinda un control fino sobre la complejidad del modelo.

Sequential Training Process of Gradient Boosting Models



Entrenamiento del Gradient Boosting

El entrenamiento de Gradient Boosting es secuencial, lo cual implica un mayor coste computacional en comparación con métodos como el bagging. Sin embargo, su desempeño suele justificar este esfuerzo.

Gracias a su capacidad para modelar relaciones no lineales y detectar interacciones entre variables, ha sido la técnica dominante en competencias como Kaggle durante varios años.

Ventajas del Gradient Boosting

1

Alta precisión

Capacidad para generar modelos altamente precisos gracias a su naturaleza secuencial y correctiva.

2

Identificación de patrones complejos

Permite identificar patrones complejos en los datos, lo que lo hace especialmente útil en problemas donde otros modelos fallan por falta de expresividad.

3

Flexibilidad

Permite ajustar numerosos hiperparámetros que influyen directamente en el rendimiento del modelo.

Hiperparámetros Ajustables



Learning Rate

Controla la velocidad de aprendizaje del modelo, permitiendo hacer correcciones más o menos



Número de árboles

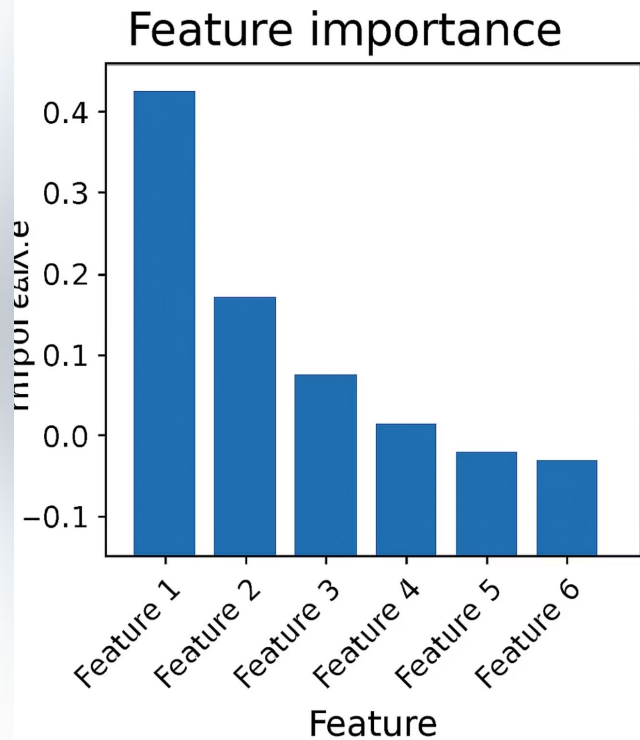
Determina cuántos modelos secuenciales se entrenarán (n_estimators).



Profundidad de árboles

Limita la complejidad de cada árbol individual (max_depth).

Esta personalización detallada posibilita adaptar el modelo a distintos tipos de datos y problemas específicos.



Versatilidad del Gradient Boosting

Este algoritmo es compatible con diversas funciones de pérdida, tanto para regresión como para clasificación. Esto lo convierte en una opción versátil para una amplia gama de tareas.

Además, permite visualizar la importancia relativa de cada variable predictora, lo que puede resultar de gran ayuda en el análisis de características y en la toma de decisiones basadas en datos.

Desventajas del Gradient Boosting

Sensibilidad al ruido

Uno de los principales retos es su sensibilidad al ruido y a los valores atípicos, ya que intenta corregir errores en cada iteración y puede terminar sobreajustando si no se regula adecuadamente.

Tiempo de entrenamiento

Al ser un método secuencial, no se beneficia tan fácilmente del paralelismo como otros algoritmos ensemble, como el bagging. Esto puede convertirse en una barrera en conjuntos de datos muy grandes o cuando se necesita respuesta en tiempo real.

Por eso, se recomienda aplicar técnicas de regularización y validación cruzada como medida preventiva.

Mitigación de Limitaciones

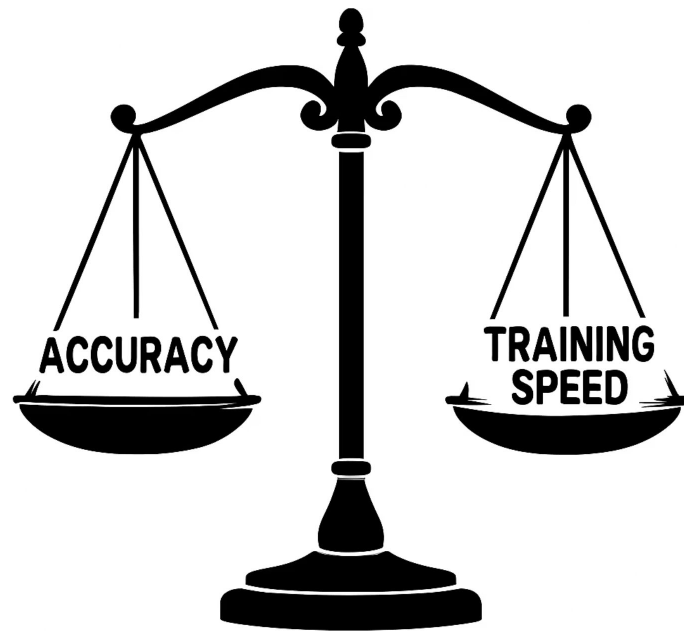
A pesar de sus limitaciones, bibliotecas optimizadas como XGBoost o LightGBM han mitigado en parte este problema mediante técnicas de procesamiento eficiente y paralelización interna.

Estas mejoras han permitido aplicar Gradient Boosting incluso en conjuntos de datos muy grandes.

Balance de Gradient Boosting

En resumen, Gradient Boosting ofrece una excelente relación entre precisión y capacidad de adaptación, lo que lo hace ideal para entornos donde se prioriza la calidad del modelo sobre la velocidad de entrenamiento.

Su implementación, aunque más compleja, está ampliamente respaldada por bibliotecas modernas que simplifican su uso y permiten aprovechar su máximo potencial.



```
import numpy as np
from sklearn.model_selection
import train_test_split
from sklearn.linear_model
import LogisticRegression

# Load and split the data
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)

# Create the model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
```

Implementación en Python

La implementación de Gradient Boosting en Python se ha facilitado enormemente gracias a bibliotecas como **Scikit-learn**, **XGBoost**, **LightGBM** y **CatBoost**.

Estas herramientas ofrecen interfaces sencillas para entrenar modelos potentes con pocas líneas de código, así como opciones avanzadas para el ajuste de hiperparámetros y evaluación del rendimiento.

Scikit-learn y Gradient Boosting

En Scikit-learn, el objeto `GradientBoostingClassifier` permite crear modelos de clasificación basados en esta técnica. Se puede configurar con múltiples parámetros como `n_estimators`, `learning_rate`, `max_depth`, entre otros, lo que proporciona flexibilidad y control sobre el comportamiento del modelo.

Además, se integra fácilmente con `GridSearchCV` para realizar búsquedas de hiperparámetros y con `Pipeline` para automatizar flujos de trabajo.

```
from sklearn.model_selection
import train_test_split

from sklearn.ensemble
import GradientBoostingClassifier

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    x, test_size=0.2, random_state=42)

# Create a Gradient Boosting classifier
gb = GradientBoostingClassifier()

# Train the classifier
gb.fit(X_train, y_train)

# Make predictions on the test set
y_pred = gb.predict(X_test)

# Evaluate the classifier
accuracy = accuracy_score(y_test, y_pred)
```

Ejemplo Práctico

A continuación se presenta un ejemplo completo utilizando el conjunto de datos de cáncer de mama disponible en Scikit-learn. El flujo incluye la carga de datos, el entrenamiento del modelo, la predicción y la evaluación con métricas estándar de clasificación:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report

# Cargar datos
data = load_breast_cancer()
X, y = data.data, data.target

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Crear y entrenar el modelo
model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
model.fit(X_train, y_train)

# Predecir y evaluar
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Precisión: {accuracy:.4f}")
print("\nInforme de clasificación:")
print(report)
```

Recomendaciones para Aplicaciones Reales

Validación cruzada

Aplicar técnicas de validación cruzada para evaluar el rendimiento del modelo de manera más robusta.

Ajuste de hiperparámetros

Utilizar GridSearchCV para encontrar la mejor combinación de hiperparámetros.

Análisis de importancia

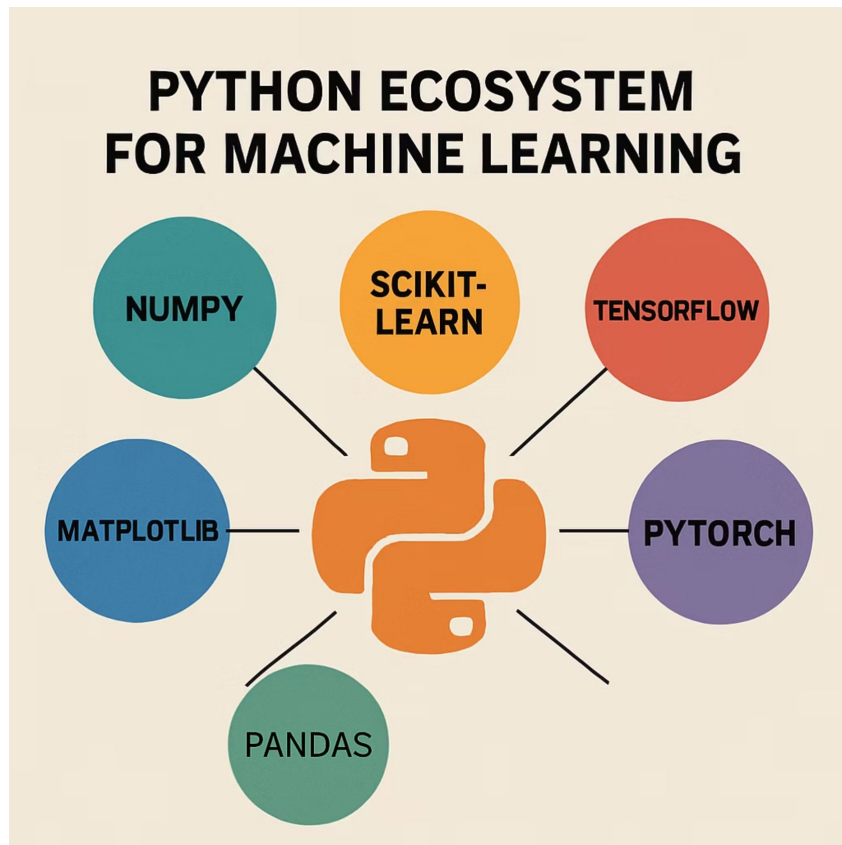
Examinar `model.feature_importances_` para interpretar el comportamiento del modelo y detectar posibles mejoras.

Este ejemplo básico puede servir como punto de partida para tareas más avanzadas.

Ventajas de la Implementación en Python

En resumen, la implementación de Gradient Boosting en Python es directa, escalable y respaldada por un ecosistema maduro de herramientas.

Esto lo convierte en una excelente opción tanto para prototipos como para soluciones listas para producción, manteniendo un equilibrio entre rendimiento y facilidad de uso.





Conclusiones sobre Gradient Boosting

El estudio de los algoritmos de boosting, en particular del Gradient Boosting, ofrece una poderosa herramienta para quienes trabajan en la ciencia de datos y el aprendizaje automático.

Su capacidad para mejorar modelos débiles de forma secuencial y enfocarse en los errores cometidos lo convierte en una técnica destacada, ideal para resolver problemas complejos de clasificación.

Live Coding

¿En qué consistirá la Demo?

Vamos a entrenar un modelo de clasificación con Gradient Boosting utilizando el dataset de cáncer de mama. Se mostrará cómo ajustar hiperparámetros e interpretar el resultado.

1. Cargar dataset `load_breast_cancer()` desde Scikit-learn.
2. Separar en train/test (80/20).
3. Crear y entrenar un modelo con `GradientBoostingClassifier`.
4. Evaluar con `accuracy_score` y `classification_report`.
5. Ajustar hiperparámetros: `n_estimators`, `learning_rate`, `max_depth`.
6. Visualizar `feature_importances_`.
7. Realizar validación cruzada con `cross_val_score`.
8. Discutir limitaciones y buenas prácticas.



Momento:

Time-out!



5 -10 min.





Ejercicio

Clasificación con Boosting

Clasificación con Boosting

Contexto: 🙌

Una startup de salud digital quiere clasificar si una paciente presenta características de cáncer benigno o maligno a partir de estudios clínicos. El equipo busca evaluar si el modelo de boosting mejora la precisión respecto a modelos anteriores.

Consigna: 📝

1. Utilizar el dataset `breast_cancer` disponible en `sklearn.datasets`.
2. Separar datos en entrenamiento y testeo.
3. Entrenar un modelo base con `GradientBoostingClassifier`.
4. Evaluar performance: precisión, matriz de confusión y F1-score.
5. Ajustar hiperparámetros usando `GridSearchCV`.
6. Graficar importancia de variables con `feature_importances_`.
7. Comparar contra un `RandomForestClassifier` en igualdad de condiciones.
8. Redactar una conclusión sobre cuál algoritmo conviene para este caso.

Paso a paso: ⚙️

1. Importar y explorar el dataset
2. Modelo inicial con boosting
3. Ajustes y comparaciones
4. Evaluación e interpretación
5. Redacción de conclusiones

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Entendimos qué es y cómo funciona el Boosting
- ✓ Comparamos Boosting con Bagging y Random Forest
- ✓ Entrenamos nuestro primer modelo de Gradient Boosting
- ✓ Ajustamos hiperparámetros para mejorar rendimiento



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

